

# From Deterministic Utilities to Cognitive Prototypes

An Applied Programming and Experimental Systems Overview

T. R. Bentley

Repository: Tybent18/practical-utilities

Document class: repository-grounded technical report

Edition: September 2026

## Scope

Applied utilities, artificial sensory abstractions, and emotion-state prototypes.

## Evidence statement

Claims in this report are limited to repository artifacts identified by filename. Proposed tests, benchmarks, integrations, and research directions are labeled as future work; they are not reported results.

# Abstract

This report separates a mixed programming laboratory into three evidence classes: deterministic command-line utilities, artificial sensory abstractions, and artificial emotion models. The separation prevents ordinary calculators and speculative cognitive components from being evaluated under the same standard. Implemented modules are cataloged, architectural relationships are proposed, and all cognitive language is explicitly bounded: these programs model variables and rules, not consciousness or human-equivalent perception.

## 1. Repository evidence map

| Cluster              | Artifacts             | Interpretation                                                                                              |
|----------------------|-----------------------|-------------------------------------------------------------------------------------------------------------|
| Everyday utilities   | C and Java root files | Financial calculation, money handling, validation, text manipulation, and bit operations.                   |
| Sensory abstractions | senses/               | Vision, hearing, touch, smell, taste, and spatial-awareness modules.                                        |
| Emotion models       | artificial_emotion/   | Focused state modules including curiosity, trust, stress, empathy, attachment, fear, and related variables. |

## 2. Technical framing

- How should deterministic tools and exploratory cognitive software be documented differently?
- What common interface could connect sensory signals, memory, emotion state, and decisions?
- Which claims require experiments rather than code inspection?

## 3. Reproducible inspection protocol

A reviewer should clone the repository, record the commit SHA and toolchain versions, inspect each mapped artifact, execute only examples whose entry points and prerequisites are declared, preserve standard output and errors, and compare observations with documented expectations. Failures in deliberate debugging targets are evidence only when the expected failure class is stated in advance.

## 4. Evidence maturity

| Level | Meaning            | Current interpretation                           |
|-------|--------------------|--------------------------------------------------|
| E0    | Artifact listed    | Source file exists in the repository             |
| E1    | Behavior described | README or report states intended behavior        |
| E2    | Execution recorded | Command, environment, and output are retained    |
| E3    | Behavior tested    | Repeatable assertions evaluate expected behavior |
| E4    | Comparative result | Frozen data supports a bounded comparison        |

## 5. Limitations

- The cognitive modules are early prototypes, not evidence of consciousness.
- The sensory modules do not yet process validated real-world multimodal datasets.
- Cross-module dynamics, tests, and measured evaluations remain future work.

## 6. Development roadmap

- Define typed input/output contracts for every cognitive module.
- Extract Adaptive Memory Dynamics as a standalone research project.
- Create frozen behavioral scenarios before making learning or adaptation claims.

## 7. Portfolio role

This repository belongs to a cumulative foundation sequence: Foundations & Algorithms → Data Structures Practice → OOP Concepts → Math for Computing → Practical Utilities. The sequence demonstrates increasing abstraction and application; advanced repositories carry the stronger systems and empirical-research claims.

## Conclusion

The repository is most credible when its modest scale is treated as a strength: each artifact can be inspected directly, every claim can point to code, and future measurements can be added without rewriting history. Sophistication comes from traceability, not inflated labels.